❄️ Snowflake FILE FORMAT

Complete & Detailed Explanation (Fresher → Experienced) only FILE FORMAT, explained **theoretically + practically + real-time examples**.

What is FILE FORMAT in Snowflake?

A **FILE FORMAT** tells Snowflake **how to read the data inside a file**.

👉 Snowflake **does NOT guess** file structure
👉 You must tell:

- File type (CSV / JSON / PARQUET)
- Column separator
- Header rows
- Quotes
- Date formats
- Null values

Simple Definition

FILE FORMAT = **Rules to read external files correctly**

Real-life example

Think of FILE FORMAT like **rules for reading a book**:

Book	File Format
Language	CSV / JSON
Line breaks	Record delimiter
Commas	Field delimiter
Quotes	Enclosed by
Header	Skip header

Why FILE FORMAT is Mandatory?

Imagine this CSV file:

id,name,city

1,Ravi,Hyderabad

2,Sita,Bangalore

Snowflake questions:

❓ Is first row header or data?
❓ Comma or pipe separator?
❓ Are values quoted?
❓ How many columns?

➡️ FILE FORMAT answers **all these questions**

Without FILE FORMAT, Snowflake does not know:

❌ Where a row starts
❌ Where a column ends
❌ Whether header exists
❌ How quotes behave
❌ How NULL values appear

So **COPY command always depends on FILE FORMAT**.

Where FILE FORMAT is Used?

FILE FORMAT is used in:

- COPY INTO
- SELECT FROM @STAGE
- VALIDATION_MODE
- External tables

FILE FORMAT used in COPY INTO (Data Load)

Purpose

Load data from stage → Snowflake table

Example

COPY INTO customers

FROM @EXTERNAL_STG/DW/loan_schema/dimcustomer/

FILE_FORMAT = CSV_FILEFORMAT_NAME;

What happens?

✔️ Snowflake reads CSV using rules from CSV_FILEFORMAT_NAME
✔️ Skips header
✔️ Handles quotes & delimiters
✔️ Loads data into customers table

📌 **Most common usage of FILE FORMAT**

FILE FORMAT used in SELECT FROM @STAGE (Preview Data)

Purpose

Preview / validate data **before loading**

Example

```
SELECT $1, $2, $3, METADATA$FILENAME
FROM @EXTERNAL_STG/DW/loan_schema/dimcustomer/
(FILE_FORMAT => CSV_FILEFORMAT);
```

Why this is important?

- ✔ Check column order
- ✔ Identify bad data
- ✔ Debug delimiter issues
- ✔ See file names

📌 **Best practice before COPY INTO**

FILE FORMAT used in VALIDATION_MODE (Error Checking)

Purpose

Check errors **without loading data**

Example

```
COPY INTO customers
FROM @EXTERNAL_STG/DW/loan_schema/dimcustomer/
FILE_FORMAT = CSV_FILEFORMAT_NAME
VALIDATION_MODE = RETURN_ALL_ERRORS;
```

What happens?

- ✔ No data is loaded
- ✔ Snowflake returns:
  - Error rows
  - Error reason
  - Line number

📌 Used in **production debugging**

FILE FORMAT used in EXTERNAL TABLES

Purpose

Query files **directly from S3** without loading

Step 1: Create External Table

```
CREATE OR REPLACE EXTERNAL TABLE ext_customers (
  id INT, first_name  STRING, last_name   STRING, email  STRING
) WITH LOCATION = @EXTERNAL_STG/DW/loan_schema/dimcustomer/
FILE_FORMAT = CSV_FILEFORMAT_NAME;
```

Step 2: Query External Table

```
SELECT * FROM ext_customers;
```

What happens?

✔ Data stays in S3

✔ Snowflake reads using FILE FORMAT

✔ No COPY required

📌 Used for data lake / ad-hoc analysis

Types of FILE FORMATS in Snowflake

Type	Used for
CSV	Most common (Excel, flat files)
JSON	Semi-structured data
PARQUET	Big data / compressed
AVRO	Streaming data
XML	Rare but supported
ORC	Hadoop ecosystems

👉 In your case: **CSV**

Creating a FILE FORMAT (Basic)

Command

CREATE FILE FORMAT CSV_FILEFORMAT_NAME

TYPE = CSV;

What happens?

✔ Snowflake now knows:

- File type is CSV
- Default delimiter is comma ,
- Default record delimiter is newline
- No header assumed

Adding Header Handling (VERY IMPORTANT)- SKIP_HEADER

Your file:

id,first_name,last_name,email

1,Ravi,Kumar,ravi@gmail.com

Problem

Snowflake will try to load:

id → INT ❌

Solution

CREATE FILE FORMAT CSV_FILEFORMAT_NAME

TYPE = CSV

SKIP_HEADER = 1;

✔ Header row ignored
✔ Only actual data loaded

SKIP_HEADER Explained Deeply

Value	Meaning
0	No header
1	Skip first row
2	Skip first 2 rows

Used when:

- Files contain titles
- Metadata rows exist

FIELD_DELIMITER (Column Separator)

Default

FIELD_DELIMITER = ','

Example file

1|Ravi|Kumar|India

Correct file format

CREATE FILE FORMAT CSV_FORMAT_NAME

TYPE = CSV

FIELD_DELIMITER = '|';

✔ Snowflake now reads | as column separator

FIELD_OPTIONALLY_ENCLOSED_BY (MOST COMMON ERROR FIX)

Problem Data

"Hyderabad, India"

Comma inside value ❌
Snowflake thinks it's two columns

Solution

FIELD_OPTIONALLY_ENCLOSED_BY = '"'

Full command

CREATE FILE FORMAT CSV_FILEFORMAT

TYPE = CSV

SKIP_HEADER = 1

FIELD_OPTIONALLY_ENCLOSED_BY = '"';

✔ Text inside quotes treated as single column

Handling NULL Values

Example file:

1,Ravi,,India

Snowflake sees empty value.

Define NULL

NULL_IF = ('', 'NULL', 'null');

✔ Empty string becomes NULL

Solution

ALTER FILE FORMAT CSV_FILEFORMAT_NAME

SET NULL_IF = ('', 'NULL', 'null');

✔ Empty values
✔ NULL / null strings
➡ All treated as **NULL** in Snowflake

EMPTY_FIELD_AS_NULL

EMPTY_FIELD_AS_NULL = TRUE;

1,Ravi,

➡ city = NULL

Solution

ALTER FILE FORMAT CSV_FILEFORMAT_NAME

SET EMPTY_FIELD_AS_NULL = TRUE;

✔ Empty field (1,Ravi,)

➡ Column value stored as NULL in Snowflake

Handling Spaces & Trimming

TRIM_SPACE = TRUE;

Example:

" Ravi "

Becomes:

Ravi

DATE_FORMAT

DATE_FORMAT = 'YYYY-MM-DD';

If file has:

2024-09-07

FULL REAL-TIME FILE FORMAT (Production Level)

CREATE OR REPLACE FILE FORMAT CSV_FILEFORMAT_NAME

TYPE = CSV

SKIP_HEADER = 1

FIELD_DELIMITER = ','

RECORD_DELIMITER = '\n'

FIELD_OPTIONALLY_ENCLOSED_BY = '"'

NULL_IF = ('', 'NULL', 'null')

TRIM_SPACE = TRUE

ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;

Describe FILE FORMAT (VERY IMPORTANT)

DESCRIBE FILE FORMAT CSV_FILEFORMAT;

Shows:

- All properties
- Default values
- Modified values

Used during:

✔ Debugging
✔ Interviews
✔ Production issues

Alter FILE FORMAT (After Creation)

You **do not recreate** file format in real projects.

```
ALTER FILE FORMAT CSV_FILEFORMAT_NAME
SET FIELD_OPTIONALLY_ENCLOSED_BY = '"';
```

- ✔ Used when data changes
- ✔ Safer than DROP & CREATE

Preview Data Using FILE FORMAT

```
SELECT
 $1, $2, $3,
 METADATA$FILENAME
FROM @EXTERNAL_STG/path/
(FILE_FORMAT => CSV_FILEFORMAT);
```

- ✔ Validate columns
- ✔ Catch errors before COPY

✅ FINAL SUMMARY

- ✔ FILE FORMAT explains **how Snowflake reads files**
- ✔ Mandatory for COPY
- ✔ Most errors come from wrong file format
- ✔ Always preview data before COPY
- ✔ Real projects **ALTER**, not recreate